

- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Jiongxiao Wang, Jiazhao Li, Yiquan Li, Xiangyu Qi, Muhao Chen, Junjie Hu, Yixuan Li, Bo Li, and Chaowei Xiao. Mitigating fine-tuning jailbreak attack with backdoor enhanced alignment. *arXiv preprint arXiv:2402.14968*, 2024.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.
- Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. Assessing the brittleness of safety alignment via pruning and low-rank modifications. *arXiv preprint arXiv:2402.05162*, 2024.
- Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. *ArXiv*, abs/2109.01652, 2021a.
- Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021b.
- Xianjun Yang, Xiao Wang, Qi Zhang, Linda Petzold, William Yang Wang, Xun Zhao, and Dahua Lin. Shadow alignment: The ease of subverting safely-aligned language models. *arXiv preprint arXiv:2310.02949*, 2023.
- Jingwei Yi, Rui Ye, Qisi Chen, Bin Zhu, Siheng Chen, Defu Lian, Guangzhong Sun, Xing Xie, and Fangzhao Wu. On the vulnerability of safety alignment in open-access llms. In *Findings of the Association for Computational Linguistics ACL 2024*, pp. 9236–9260, 2024a.
- Xin Yi, Shunfan Zheng, Linlin Wang, Xiaoling Wang, and Liang He. A safety realignment framework via subspace-oriented model fusion for large language models. *arXiv preprint arXiv:2405.09055*, 2024b.
- Wenxuan Zhang, Philip HS Torr, Mohamed Elhoseiny, and Adel Bibi. Bi-factorial preference optimization: Balancing safety-helpfulness in language models. *arXiv preprint arXiv:2408.15313*, 2024.
- Wei Zhao, Zhe Li, Yige Li, Ye Zhang, and Jun Sun. Defending large language models against jailbreak attacks via layer-specific editing. *arXiv preprint arXiv:2405.18166*, 2024.
- Yongshuo Zong, Ondrej Bohdal, Tingyang Yu, Yongxin Yang, and Timothy Hospedales. Safety fine-tuning at (almost) no cost: A baseline for vision large language models. *arXiv preprint arXiv:2402.02207*, 2024.
- Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023.

A APPENDIX

A.1 TEMPLATE OF INSTRUCTION TUNING AND INFERENCE

Throughout the instruction-tuning and inference process, we adopt a template format based on the guidelines provided by Taori et al. (2023). This template can be classified into two types: with input and without input, as illustrated below:

Templates with and without input	
<i>Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.</i> ### Instruction: {instruction} ### Input: {input} ### Response: {output}	<i>Below is an instruction that describes a task. Write a response that appropriately completes the request.</i> ### Instruction: {instruction} ### Response: {output}

During inference, the “output” key of the data is set empty. After inserting different questions in the template during inference, the last position of the token is always ‘:’, so that LLM can perform next word prediction.

The fine-tuning data is structured with three keys: “Instruction”, “Input”, and “Output” (Wei et al., 2021a; Ouyang et al., 2022a). “Instruction” defines the task, “Input” complements it, and “Output” holds answers and explanations. In instruction tuning, we categorize the data based on the presence or absence of the “input” key and insert each category into their respective templates as fine-tuning data for the LLM.

A.2 MORE RELATED WORKS

Since (Qi et al., 2023) demonstrates the fine-tuning risk, there aligns with the defenses coming out trying to solve the issue. Here, we categorize these studies to three stages: the alignment stage, the fine-tuning stage, and the post-fine-tuning stage.

Alignment stage. Vaccine(Huang et al., 2024c) creates invariant embeddings through incremental perturbations during alignment, enabling resistance to harmful user data in fine-tuning. Rep-Noise(Rosati et al., 2024) removes harmful representations to prevent recovery during fine-tuning, even with attacker access to model weights. STAGE uses data management to enhance LLM security by modifying commonsense text. TAR(Tamirisa et al., 2024) builds tamper-resistant protections into open LLMs, surviving thousands of fine-tuning steps. Finally, DEEP suggests deepening safety alignment beyond initial tokens to improve robustness against common exploits.

Fine-tuning stage. Several approaches focus on fine-tuning while preserving model safety. LD-IFS (Mukhoti et al., 2023) retains pre-trained knowledge while learning new tasks, with experiments showing that adding just 3% of safe examples significantly improves safety (Bianchi et al., 2023). Lisa (Huang et al., 2024b) bounds state drift to prevent rapid safety degradation, while BFPO Zhang et al. (2024) reparameterizes RLHF objectives into a single supervised learning framework.

For prompt-level fine-tuning, Wang et al. (2024) adds security hints to backdoor data, enabling maliciously fine-tuned LLMs to achieve security similar to aligned models. Lyu et al. (2024) introduces the “Pure Tuning, Safe Testing” (PTST) principle, fine-tuning without safety prompts but applying them at test time.

In multimodal models, fine-tuning can also reduce safety. VLGuard (Zong et al., 2024), a defense dataset, effectively protects vision-language models when used during fine-tuning.

Post-fine-tuning stage. In this domain, Yi et al. (2024b) proposes SOMF, combining the safety of the original and fine-tuned models into a re-aligned model. Safe LoRA(Hsu et al., 2024) reduces security risks by projecting selected layers into a safe subspace. Antidote (Huang et al., 2024a) prunes harmful parameters post-fine-tuning to recover model safety. Leong et al. (2024) outlines a three-phase protection process: recognizing harmful commands, generating rejection tones, and completing responses. Safety Basin(Peng et al., 2024) shows random weight perturbations maintain safety locally but cause sharp declines beyond this region.

Moreover, in our concurrent work, Zhao et al. (2024) also introduced the concept of safety layers. However, despite the shared terminology, the definition, identification method, and purpose of their safety layers differ significantly from ours. Specifically, Zhao et al. (2024) identifies safety layers by pruning various layers of the LLM and evaluating the security degradation in the pruned model. The top-k layers most closely associated with security are defined as safety layers. Based on this identification, they designed the LED algorithm, which performs knowledge editing on these safety layers and additional auxiliary layers during inference to defend against adversarial attacks.

In contrast, our approach identifies safety layers through an analysis of how aligned LLMs process different input vectors. In terms of application, our method targets fine-tuning jailbreak scenarios for LLMs and does not rely on additional auxiliary layers unrelated to security. Furthermore, the safety layer ranges identified in their experiments (e.g., for LLaMA-2-Chat) differ entirely from those identified in our work.

In summary, while the safety layers introduced by Zhao et al. (2024) share the same name as our concept, they are distinct in their definition, methodology, and application. Nevertheless, their work provides another interesting perspective for layer-wise analysis of LLM security.

A.3 SAFETY LAYERS: LOCATING

A.3.1 LAYER-WISE VECTOR COSINE SIMILARITY ANALYSIS FOR OTHER ALIGNED LLMs

Graphs about Llama-2-7b-chat, Phi-3-mini-4k-instruct, gemma-2b-it are in figure 6, figure1 and figure 7. These graphs illustrate that these three sets of curves are inherent to the LLM itself and are not altered when different sentences in each analysis are selected. Replacing sentences with different semantics shows minimal fluctuations in each of these curves indicating the widespread presence of these three properties on aligned LLMs.

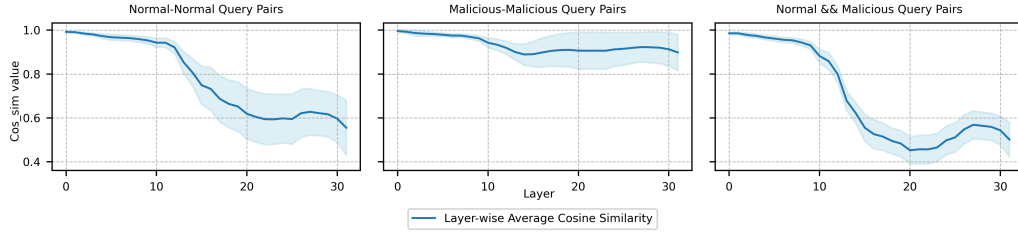


Figure 5: The mean cosine similarity of the final position vector for each layer in **Llama-3-8B-Instruct** when exposed to Normal-normal, Malicious-malicious, and Normal-malicious question pairs during inference. The shaded region represents the fluctuation range of the cosine similarity curve at each analysis setting, which arises from the random selection within each problem set. Statistical calculations were performed with the settings $P = 100$, $Q = 100$, and $r = 500$.

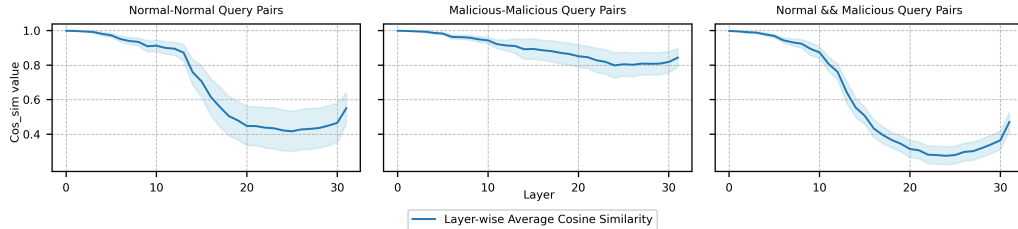


Figure 6: The mean cosine similarity of the final position vector for each layer in **Llama-2-7b-chat** when exposed to Normal-normal, Malicious-malicious, and Normal-malicious question pairs during inference. The shaded region represents the fluctuation range of the cosine similarity curve at each analysis setting, which arises from the random selection within each problem set. Statistical calculations were performed with the settings $P = 100$, $Q = 100$, and $r = 500$.

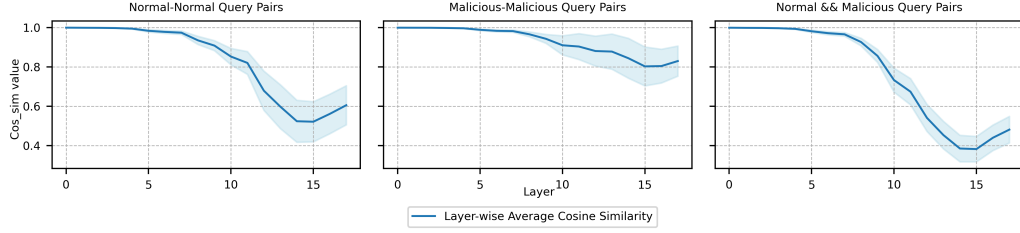


Figure 7: The mean cosine similarity of the final position vector for each layer in **gemma-2b-it** when exposed to Normal-normal, Malicious-malicious, and Normal-malicious question pairs during inference. The shaded region represents the fluctuation range of the cosine similarity curve at each analysis setting, which arises from the random selection within each problem set. Statistical calculations were performed with the settings $P = 100$, $Q = 100$, and $r = 500$.

A.3.2 DIFFERENT PROMPT TEMPLATE

Our cosine similarity analysis of N-N pairs and N-M pairs is valid for different prompt templates. Different prompt templates are similar in structure to the alpaca template we used, differing only in content. In the main text, we consistently used the alpaca template across different aligned LLMs analyses to standardize the experimental setup.

Further, we applied the default chat template for each aligned LLM and re-plotted the layer-wise analysis of safety layer existence in figure 8.

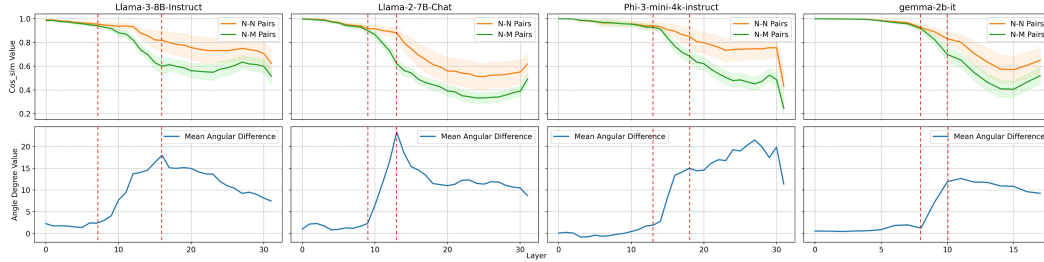


Figure 8: The upper half shows the “Normal-Normal(N-N) Pairs” and “Normal-Malicious(N-M) Pairs” cosine similarity analysis results for each hidden layer of Llama-3-8B-Instruct, Llama-2-7B-Chat, Phi-3-mini-4k-instruct and gemma-2b-it. The lower half displays the mean angular difference between these two cases for each aligned LLM. The default templates of each aligned LLMs were used in the analysis process.

Comparing these figures to the safety layer existence analysis graph using the Alpaca prompt template (Figure 2 in the main text), we observed that switching to the default prompt template caused only minor changes in the cosine similarity curve values, without significantly altering the overall curve trends. Moreover, the layers where the N-N and N-M curve gap emerges and converges (marked by the red dashed lines) remained largely consistent across both prompt templates.

From the perspective of our method’s design, the cosine similarity analysis of different question pairs is aimed at visualizing how the model processes and distinguishes normal and malicious questions across its internal layers. Regardless of the prompt template used for instructions, aligned LLMs can always classify whether to refuse a response solely based on the content of the instruction. Further, as long as the internal layers of the model can differentiate between malicious and normal questions, our method can illustrate the role of the safety layers through the gap in the curve. Consequently, the choice of prompt template does not impact our analysis or identification of safety layers, making our method broadly applicable across various prompt templates.